Stereo-Vision Closed-Loop Manipulation with Open-Vocabulary Grounding

Yingjun Lan Jinhao Zhang Zhengxiang Liu

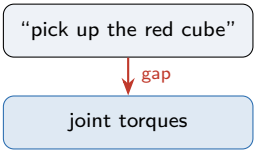
School of Artificial Intelligence, Shanghai Jiao Tong University

Final Project Presentation

The language $\rightarrow$ action gap

Goal. From a free-form instruction to robot joint torques.

- Instruction lives in a semantic space.
- Control lives in a metric space.
- End-to-end VLAs (RT-2) learn the map directly...
- ...but need large teleoperated datasets and generalize poorly out of distribution.



Core idea: decompose, don't learn end-to-end

Route a frozen VLM through geometry and a symbolic planner.

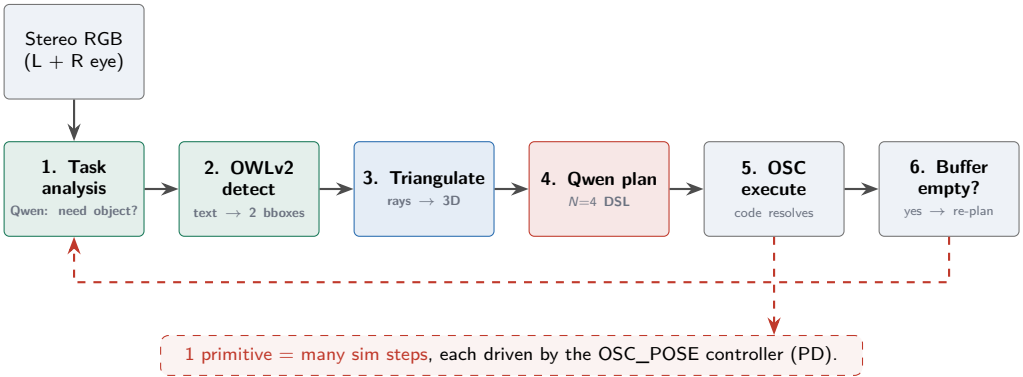
Instead of one big learned map:

- **Perception** — OWLv2 grounds the named object.
- **Geometry** — stereo triangulation $\rightarrow$ metric 3D.
- **Planning** — VLM emits a 7-primitive DSL.
- **Control** — OSC executor runs it.

Why this matters for a small team:

- Training-free — no teleoperation data.
- Inspectable: each stage is auditable.
- Runs on a single T4 GPU.

Workflow — closed-loop cycle



Open-vocabulary grounding with OWLv2

Problem. Asking the VLM to regress boxes is unreliable for small objects.

- Ground the target with OWLv2 (`owlv2-base-patch16-ensemble`, $\sim$0.6B).
- Query text comes from the instruction (target noun) or a per-trial override.
- Runs once per round on both eyes with the same query.
- Returns highest-scoring box $[x_1, y_1, x_2, y_2]$ per eye; threshold 0.05.
- Round fails only if *both* eyes miss; single-eye miss still triangulates.

Decoupling: the VLM understands *what* to grasp; OWLv2 finds *where*; geometry recovers *how far*.

Binocular stereo & ray triangulation

Setup. Inject `left_eye` / `right_eye` into MuJoCo, 0.50 m baseline, look-at table center. Back-project each box center to a viewing ray:

$$\mathbf{d}_c = R_c\big[(u-c_x)/f_x,\ (v-c_y)/f_y,\ -1\big]^\top, \quad \mathbf{r}_c(t) = \mathbf{o}_c + t\,\hat{\mathbf{d}}_c$$

Closest point of two rays (least squares):

$$(t^\star, s^\star) = \arg\min_{t,s} \left\|\mathbf{o}_L + t\hat{\mathbf{d}}_L - \mathbf{o}_R - s\hat{\mathbf{d}}_R\right\|^2, \quad \mathbf{p} = \tfrac{1}{2}\big(\mathbf{r}_L(t^\star) + \mathbf{r}_R(s^\star)\big)$$

- Residual gap $> 5\,\text{cm} \Rightarrow$ reject & re-query (free consistency check).
- Generalizes to objects *off* the reference plane (e.g. inside a bin), no table-height assumption.

RMDL: a seven-primitive motion DSL

Primitive	Parameters	Target resolution
`move_to`	target {object, above_object}, height?	code: `object_3d` ($\pm$ height)
`move_by`	offset $[dx, dy, dz]$, speed	code: $\mathbf{p}_{ee}+$ offset
`descend`	—	code: vertical to table contact (plateau)
`rotate`	rotation $[r, p, y] \in [-1, 1]$, speed	code: relative EE orientation
`grasp`	—	close gripper (sticky)
`release`	—	open gripper (guarded: only over bin)
`wait`	—	no-op (fixed duration)

What the language model outputs

The VLM returns one JSON object: a natural-language `reasoning` trace + an `actions` list of exactly $N{=}4$ DSL primitives.

```
{
 "reasoning": "Both eyes show the
  red cube front-center; 3D known.
  Approach, descend, grasp, lift
  to clear the bin wall.",
 "actions": [
  {"type":"move_to",
   "target":"above_object","height":0.05},
  {"type":"descend"},
  {"type":"grasp"},
  {"type":"move_by",
   "offset": [0,0,0.15],"speed":0.5}
 ]
}
```

Why this works:

- Validated: malformed JSON / empty array / `error` field → retry with a targeted hint (up to 10×).
- No arithmetic: `target` is a *keyword* (`object`), not a coordinate — code supplies XYZ.
- Strategy, not control: the VLM chooses *which* primitive; the executor chooses *how far*.
- Deterministic grounding: OWLv2 (not the VLM) finds the box, so the plan is anchored to a real pixel.

Execution: many steps per cycle

One cycle = the VLM plans N=4 primitives → execute them → re-plan. Each primitive runs as many sim steps as needed (until within 5 mm of its target or it plateaus), so a cycle is dozens of steps.

Execution uses OSC, not a model. Every step is driven by Robosuite's OSC_POSE controller (PD); Qwen3-VL and OWLv2 run *only* at the start of a cycle.

Inside one sim step (no neural net): read EE pose → OSC action $\Delta = \text{speed} \cdot (\text{target} - \text{ee}) +$ sticky gripper → `env.step` → check success.

History snapshots. After each primitive finishes, a stereo (L+R) frame is saved (up to 2); at re-plan the VLM sees the current views *plus* these snapshots.

Viewpoints study and result

Viewpoint study. We tried single-view, dual-view, and binocular stereo. Dual-view already beats single-view, and binocular stereo is the most practical one — so we adopt it.

Result

Binocular stereo reaches **83%** over 100 trials.

Demonstrations

- **Pick & place: red cube → wooden bin**
  grasp the cube, lift over the bin wall, drop inside, release.
- **Color transfer: red / green / blue cubes**
  the same OWLv2 + stereo + VLM pipeline handles all colours without retraining.
- **Failure case: rotating grasp of bottle & milk**
  the gripper rotates to align with non-cylindrical objects — shape or occlusion causes failure.

Demonstration - Free-motion “dance”

RL branch: GRPO-style LoRA fine-tuning of the planner

The planner *is* the policy. Rather than a separate action head, Qwen3-VL-4B itself is the policy: it emits DSL plans (`move_to`/`descend`/`grasp`/`move_by`/`release`) that the same `VLMController` executes with OWLv2 + stereo + the closed loop.

Method

- Only LoRA of Qwen is trained (r=16, α=32, all-linear); the base model stays 4-bit frozen.
- GRPO-style: each update fixes the scene seed, samples `grpo_group_size=4` candidate plans, executes each, and updates with the group-relative advantage.
- Reward = task success (binary 1/0) on pick-and-place.

Config & status

- Small sample + binary reward $\Rightarrow$ high variance; logprob is recomputed post-hoc (not textbook GRPO).
- No ideal final curve yet, and limited by hardware, but the pipeline proved feasible.
- Can be used to finish more difficult tasks in future work.

Generalization: harder tasks the framework already enables

The decoupled *what* / *where* / *how-far* design plus the re-planning loop make these reachable with no architectural change — only detection targets, prompts, or camera placement change.

- **Obstacle-aware pick-and-place** (easy) — add the obstacle as a second OWLv2 target and one prompt rule; the VLM plans a detour before placing.
- **Shape-aware grasp** (hard, prompt-only) — e.g. grasp a cup by its *handle*. The VLM already perceives shape; a grasp-pose prompt lets it choose *where* to grip, while geometry supplies the 3D.
- **Escaping the binocular blind spot** (camera swap) — if OWLv2 misses, let the VLM steer the arm to search; a *wrist-view* camera makes 3D recovery a simple pose-transform of the same ray triangulation.

Each is a consequence of the same modular stack — not new learning.

Conclusion

Takeaway. A frozen VLM + open-vocabulary detector + binocular stereo, glued by a 7-primitive DSL and a re-planning loop, performs training-free language-guided manipulation.

Thank you

References

Selected references

- Mandlekar et al. *Robomimic*, CoRL 2021.
- Brohan et al. *RT-2*, CoRL 2023.
- Gu et al. *OWLv2* (scaling open-vocabulary detection), CVPR 2024.
- Qwen Team. *Qwen3-VL*, 2025.
- Zhu et al. *robosuite*, arXiv 2020.
- Mittal et al. *Isaac Lab / Isaac Sim*, 2023.